

Revisão de estrutura de árvores

(Atividade individual remota para o dia 28/09)

Nome: Isabela Linhares de Oliveira

Disciplina: Estrutura de Dados II Profa. Kadidja Valéria

Modalidade: individual, remota e assíncrona **Tempo estimado:** 1 hora e 45 minutos

A atividade pode ser realizada integralmente de casa. Não é necessário comparecer à faculdade.

Objetivos de aprendizagem

- Retomar os conceitos de nó, raiz, pai, filho, folha, altura e percurso.
- Diferenciar as regras e aplicações dos tipos de árvores estudados.
- Relacionar analogias cotidianas às propriedades técnicas de cada estrutura.

Etapa 1 — Revisão bibliográfica (35 minutos)

1. Conceitos Básicos de Árvore

Uma árvore é uma estrutura de dados não linear composta por um conjunto de nós (ou vértices) e arestas que os conectam hierarquicamente.

- **Nó Raiz:** O único nó superior que não possui pai (ponto de entrada da árvore).
- **Nó Pai e Filho:** Relação hierárquica direta onde um nó aponta para outro abaixo dele.
- **Nó Folha:** Nó que não possui nenhum filho.
- **Altura:** O comprimento do caminho mais longo da raiz até uma folha (ou o número de arestas desse caminho).
- **Percurso:** Forma de visitar todos os nós da árvore de maneira sistemática (ex.: pré-ordem, em-ordem, pós-ordem e em largura).

2. Árvore Geral

- **Organização dos dados:** Cada nó pode possuir um número arbitrário (ilimitado) de filhos.
- **Propriedade mantida:** Relação hierárquica livre de ciclos, ideal para representar hierarquias organizacionais ou sistemas de ficheiros.
- **Busca e Inserção:** A busca exige percorrer os ramos de forma exaustiva (como pesquisa em largura ou profundidade) se não houver ordenação prévia. A inserção é feita anexando o novo nó como filho do nó pai pretendido.
- **Ajustes:** Geralmente não possui restrições de balanceamento automático nativas, podendo exigir reestruturações manuais dependendo da aplicação.
- **Contexto de utilidade:** Representação de sistemas de diretórios (pastas e subpastas) e árvores sintáticas em compiladores.

3. Árvore Binária

- **Organização dos dados:** Cada nó possui, no máximo, dois filhos, denominados filho esquerdo e filho direito.
- **Propriedade mantida:** Grau máximo de ramificação igual a 2 por nó.
- **Busca e Inserção:** A busca não possui ordenação garantida por defeito, exigindo percurso completo no pior

caso $O(N)$. A inserção é feita no primeiro espaço disponível ou conforme a lógica da árvore específica.

- **Ajustes:** Sem rebalanceamento, a árvore pode degenerar para uma estrutura semelhante a uma lista ligada.
- **Contexto de utilidade:** Estrutura base para algoritmos mais complexos, árvores de expressão matemática e árvores de decisão.

4. Árvore Binária de Busca (ABB / BST)

- **Organização dos dados:** Os dados são dispostos de forma que, para qualquer nó, todos os valores da subárvore esquerda são menores do que o nó, e os da subárvore direita são maiores.
- **Propriedade mantida:** $\text{Valor(esquerda)} < \text{Valor(raiz)} < \text{Valor(direita)}$.
- **Busca e Inserção:** A busca compara a chave procurada com o nó atual, descendo para a esquerda ou direita de forma logarítmica $O(\log N)$ no caso médio. A inserção localiza a posição correta respeitando a desigualdade e insere como nova folha.
- **Ajustes:** Pode ficar desbalanceada (desgenerada) em inserções ordenadas, fazendo com que o desempenho caia para $O(N)$.
- **Contexto de utilidade:** Tabelas de símbolos e dicionários dinâmicos onde a simplicidade de implementação é valorizada e os dados chegam de forma razoavelmente aleatória.

5. Árvore AVL

- **Organização dos dados:** Uma árvore binária de busca auto-balanceada.
- **Propriedade mantida:** Para qualquer nó, a diferença de altura entre a subárvore esquerda e a subárvore direita (fator de balanceamento) deve ser no máximo 1 ($BF < 1$).
- **Busca e Inserção:** A busca é idêntica à da ABB ($O(\log N)$). A inserção pode violar a propriedade de balanceamento, exigindo validação a partir do nó inserido até a raiz.
- **Ajustes:** Realiza rotações simples (à esquerda ou à direita) e duplas (esquerda-direita ou direita-esquerda) para restabelecer o equilíbrio após inserções ou remoções.
- **Contexto de utilidade:** Aplicações que exigem um número elevado de operações de busca intensiva com garantia estrita de tempo de resposta logarítmico.

6. Árvore Rubro-Negra (Red-Black Tree)

- **Organização dos dados:** Árvore binária de busca onde cada nó armazena um bit adicional de cor (rubro ou negro).
- **Propriedade mantida:**
 - Todo nó é vermelho ou preto.
 - A raiz é preta.
 - Todas as folhas (NIL) são pretas.
 - Se um nó for vermelho, os seus filhos devem ser pretos (sem dois vermelhos seguidos).
 - Todo caminho de um nó até as suas folhas descende o mesmo número de nós pretos.
- **Busca e Inserção:** A busca segue a regra da ABB. A inserção insere nós como vermelhos e pode violar as regras de cores.
- **Ajustes:** Utiliza recolorações de nós e rotações para manter o equilíbrio aproximado, garantindo que o caminho mais longo não seja o dobro do mais curto.
- **Contexto de utilidade:** Implementação de bibliotecas padrão em linguagens de programação (como `std::map`

e `std::set` no C++ ou `TreeMap` em Java).

7. Árvore B

- **Organização dos dados:** Árvore de busca balanceada auto-ajustável voltada para sistemas de armazenamento secundário (discos rígidos), onde cada nó pode conter múltiplos filhos e chaves.
- **Propriedade mantida:** Todos os nós folhas encontram-se na mesma profundidade; cada nó possui um limite máximo e mínimo de filhos determinado por uma ordem m .
- **Busca e Inserção:** A busca percorre as chaves internas de um nó (frequentemente usando busca binária interna) para decidir o ponteiro descendente. A inserção é feita no nó folha; se exceder a capacidade máxima, ocorre uma divisão (*split*) da chave mediana para o pai.
- **Ajustes:** Divisões e fusões de nós para manter o balanceamento perfeitamente plano.
- **Contexto de utilidade:** Sistemas de gestão de bases de dados (SGBD) e sistemas de ficheiros para indexação em disco.

8. Árvore B+

- **Organização dos dados:** Variação da Árvore B onde todos os dados/registros reais (ou ponteiros diretos para eles) residem exclusivamente nos nós folhas, enquanto os nós internos armazenam apenas chaves de roteamento.
- **Propriedade mantida:** As folhas são interligadas sequencialmente por meio de ponteiros (formando uma lista ligada nas folhas).
- **Busca e Inserção:** A busca navega pelos nós internos de forma idêntica à Árvore B até atingir a folha desejada. A inserção reorganiza chaves internas e folhas com divisões quando necessário.
- **Ajustes:** Divisão de nós e manutenção dos ponteiros sequenciais nas folhas.
- **Contexto de utilidade:** Indexação de bases de dados e sistemas de ficheiros onde consultas por intervalo (ex.: "encontrar todos os registos entre a data X e a data Y") são frequentes e eficientes.

9. Heap (Árvore de Prioridade)

- **Organização dos dados:** Uma árvore binária quase completa armazenada tipicamente num vetor.
- **Propriedade mantida:** Na *Max-Heap*, o valor de cada pai é maior ou igual ao dos seus filhos; na *Min-Heap*, o pai é menor ou igual.
- **Busca e Inserção:** A busca por um elemento arbitrário é lenta $O(N)$, mas aceder ao elemento prioritário (raiz) é $O(1)$. A inserção coloca o elemento no final e promove-o (*bubble-up*) se violar a propriedade.
- **Ajustes:** Reorganização por subidas ou descidas (*heapify*) para restaurar a ordem após remoção da raiz ou nova inserção.
- **Contexto de utilidade:** Implementação de filas de prioridade e no algoritmo de ordenação *Heapsort*.

10. Árvore Trie (Árvore de Prefixos)

- **Organização dos dados:** Árvore de busca otimizada para recuperação de cadeias de caracteres (strings), onde cada caractere de uma palavra representa um nível ou aresta da árvore.
- **Propriedade mantida:** Os nós filhos de um nó compartilham o mesmo prefixo comum correspondente ao caminho desde a raiz.
- **Busca e Inserção:** A busca percorre os caracteres sequencialmente, com complexidade proporcional ao tamanho da palavra procurada $O(M)$ e independente do número total de palavras. A inserção adiciona nós para os caracteres faltantes da palavra.
- **Ajustes:** Não requer rotações de balanceamento estrutural complexas, mas consome memória adicional para

os ponteiros de caracteres.

- **Contexto de utilidade:** Sistemas de preenchimento automático (*autocomplete*), corretores ortográficos e dicionários de palavras.

Etapa 2 — Quadro comparativo (35 minutos)

Preencha todas as linhas pertinentes ao conteúdo da disciplina. Acrescente árvore geral, árvore binária, heap e trie quando forem abordadas nas referências ou aulas. A coluna de referência deve apontar a fonte específica usada para cada linha.

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação	Referência
ABB	Nós organizados em hierarquia binária com subárvores restritas por valor.	Esq < Raiz < Dir	Inserção como folha; busca sequencial binária; sem ajustes automáticos de altura.	Dicionários simples e tabelas de símbolos.	Material de Aula / Cormen et al.
AVL	Árvore binária de busca com controlo estrito de balanceamento de alturas.	Diferença de altura entre subárvores esquerda e direita de qualquer nó < 1.	Rotações simples e duplas executadas após inserções/remoções.	Bases de dados em memória e dicionários de acesso rápido estrito.	Notas de Aula / Weiss, Mark Allen
Rubro-negra	Árvore binária de busca com atributos de cor (vermelho ou preto) por nó.	Regras rígidas de alternância de cores e caminhos pretos equidistantes.	Recolorações e rotações para limitar a altura desproporcional.	Bibliotecas padrão (ex: <code>std::map</code> em C++).	Cormen et al., <i>Introduction to Algorithms</i>
B	Árvore multi-caminho balanceada orientada a blocos de memória secundária.	Todos os nós folha no mesmo nível; limite de chaves por nó entre $t-1$ e $2t-1$.	Divisão (<i>split</i>) de nós cheios e fusão em remoções.	Sistemas de Ficheiros e indexação de SGBD em disco.	Silberschatz et al., <i>Database System Concepts</i>
B+	Árvore multi-caminho com dados guardados estritamente nas folhas encadeadas.	Chaves internas apenas encaminham; folhas ligadas por ponteiros sequenciais.	Divisão de folhas e propagação de chaves para nós internos.	Consultas por intervalo em bases de dados relacionais.	Elmasri & Navathe, <i>Sistemas de Bases de Dados</i>

Etapa 3 — Identificação por analogias (35 minutos)

1. **Situação:** Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.
 - **Estrutura:** Árvore AVL (ou Árvore Rubro-Negra).
 - **Propriedade técnica:** Utiliza o fator de balanceamento ou regras de altura rigorosas, aplicando rotações (simples ou duplas) para corrigir desníveis.
 - **Limite da analogia:** Na prática computacional, os dados não mudam de prateleira fisicamente; apenas os ponteiros de memória que ligam os nós são alterados.
2. **Situação:** Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.
 - **Estrutura:** Árvore B.
 - **Propriedade técnica:** Cada nó suporta um número máximo e mínimo de chaves e filhos, realizando a divisão (*split*) do nó central quando a capacidade excede o limite.
 - **Limite da analogia:** No computador, as "páginas" representam blocos de memória ou setores de disco rígido de tamanho fixo em bytes, e não páginas de papel legíveis por humanos.
3. **Situação:** Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.
 - **Estrutura:** Heap (Árvore de Prioridade / Max-Heap ou Min-Heap).

- **Propriedade técnica:** A propriedade de heap assegura que o elemento de maior (ou menor) valor esteja sempre localizado na raiz.
 - **Limite da analogia:** Embora seja conceptualmente visualizada como uma árvore binária quase completa, a sua implementação física eficiente é feita através de um vetor unidimensional (array).
4. **Situação:** Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.
- **Estrutura:** Árvore Trie (Árvore de Prefixos).
 - **Propriedade técnica:** Os nós representam caracteres individuais e as ramificações derivam do prefixo comum compartilhado pelas chaves textuais inseridas.
 - **Limite da analogia:** Uma Trie pode consumir uma quantidade excessiva de memória em nós esparsos caso existam poucas palavras com prefixos comuns longos.
5. **Situação:** Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.
- **Estrutura:** Árvore Rubro-Negra.
 - **Propriedade técnica:** Utiliza propriedades baseadas em atributos de cores (rubro e negro) associados a restrições de caminhos pretos para garantir o balanceamento aproximado.
 - **Limite da analogia:** As cores são apenas metadados lógicos (bits de controlo guardados em cada nó) e não possuem correspondência visual ou estética real.
6. **Situação:** Um índice conduz às folhas que contêm os registos, ligadas entre si para facilitar consultas por intervalo.
- **Estrutura:** Árvore B+.
 - **Propriedade técnica:** Os nós internos servem estritamente para encaminhamento, enquanto os nós folha contêm os dados e estão encadeados sequencialmente por ponteiros.
 - **Limite da analogia:** Numa árvore B+ tradicional, os dados duplicam-se parcialmente entre as chaves de roteamento internas e os valores armazenados nas folhas, o que consome mais espaço do que uma árvore B pura.
7. **Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.**
- **Estrutura:** Árvore Binária de Busca (ABB).
 - **Propriedade técnica:** Mantém a invariante estrutural onde qualquer subárvore esquerda possui chaves inferiores à do nó raiz, e a subárvore direita possui chaves superiores.
 - **Limite da analogia:** A analogia assume uma distribuição equilibrada dos dados; contudo, se os números forem inseridos de forma ordenada, a árvore degrada-se e perde a eficiência logarítmica, assemelhando-se a uma lista sequencial.

Referências

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
- Weiss, M. A. *Data Structures and Algorithm Analysis in Java / C++*. Pearson.
- Material didático e notas de aula da disciplina de *Estrutura de Dados II* - Profa. Kadidja Valéria (UDF).